Preliminary Study Task Plan

Computation-Aware Control for Software-Defined Vehicles

ARMS Lab, The University of Texas at Dallas • Advisor: Prof. Zejiang Wang • July 2026

1. Purpose and Context

The research theme is computation-aware control for software-defined vehicles (SDVs): perception, planning, and control tasks share one centralized processor, so the processor's timing, energy, and thermal behavior become part of the closed-loop dynamics rather than an implementation detail.

Your job is to build the **virtual co-design testbed** — a modular Simulink environment coupling a vehicle plant, an MPC controller, a task scheduler, and (later) power/thermal dynamics — and to produce from it the preliminary evidence for two proposals, including four proposal-quality figures, one optional hardware figure, and a fully reproducible codebase.

The testbed is the central artifact. Every figure is an output of the testbed; the testbed itself is a deliverable that outlives this study and becomes the lab's research infrastructure.

The summer portion (Phases 0–3) is a mutual evaluation period. In the first week of September we will review your deliverables together and decide whether to proceed with the funded studentship. The evaluation criteria and the deliverable checklist are written explicitly in Section 8, so you know the bar in advance.

2. Working Rules

- **Weekly meeting.** Every Monday from 07/27, one report summarizes what you did, what you learned, what blocked you, and plan for the next week. These reports become proposal text; writing quality matters.
- **One repository, everything reproducible.** All code in Git. Every figure must be regenerable by one named script from a clean clone. A figure that cannot be regenerated does not exist.
- **Figure standard.** Labeled axes with units, captions, export to .fig. Fixed random seeds; Monte Carlo results as mean with shaded bands.
- **Freeze the controller after Phase 0.** MPC weights, horizon, constraints, and internal model are frozen for the entire study. Retuning per experiment destroys cross-phase comparability.
- **Log solver time always.** Every closed-loop run logs the MPC solve time at every step, from Phase 0 onward. This telemetry is not optional bookkeeping — it becomes the execution-time model in Phase 2.
- **The advisor's vehicle model is read-only.** Use the provided mid-fidelity Simulink model as the plant.
- **Escalate after two days.** Any tooling problem (licensing, compilation, solver interfaces) that blocks you for more than two working days goes to the advisor with a precise description of what you tried.
- **Do not touch dSPACE.** VEOS and SCALEXIO are post-award work. Nothing before February 2027 requires them.

3. Reading List

For each item, your report states the main idea, one thing that surprised you, and one question it raised

#	Item	Read during	What to extract
1	W. Chang, *Resource-Aware Automotive Control Systems Design: A Cyber-Physical Systems Approach* (PhD dissertation, TU Munich)	Phase 0	The closest prior art to this project. Map which co-design levers it covers and — more importantly — which it does not (thermal/DVFS coupling, SDV consolidation, scenario-dependent MPC compute). That gap map is proposal material.
2	Årzén, Cervin, Eker, Sha, "An Introduction to Control and Scheduling Co-Design," IEEE CDC 2000	Phase 0	The whole problem in six pages: why control and scheduling cannot be designed separately.
3	Cervin et al., "How Does Control Timing Affect Performance?" IEEE Control Systems Magazine 2003	Phase 1	Vocabulary (latency, jitter, response time) and the evaluation methodology. Ignore the TrueTime tooling; keep the methodology.

4	Buttazzo, *Hard Real-Time Computing Systems*, Ch. 1–4	Phase 2	Task model (period, deadline, WCET), fixed-priority scheduling, utilization bounds, response-time analysis.
5	Chakraborty et al., "Automotive Cyber-Physical Systems: A Tutorial Introduction," IEEE Design & Test 2016	Phase 2	How automotive software is actually scheduled (fixed priorities, AUTOSAR practice) — why our testbed defaults are realistic.
6	Gros, Zanon, Quirynen, Bemporad, Diehl, "From Linear to Nonlinear MPC: Bridging the Gap via the Real-Time Iteration," Int. J. Control 2020	Phase 1–2	The standard compute-aware NMPC scheme (RTI, which ACADOS implements): how MPC is made to live inside a fixed compute budget.

4. Phase 0 — Testbed Foundation: Plant + MPC Baseline (July)

Objective. Stand up the plant–controller core that every later phase reuses, and start the solver-time telemetry. Nothing here is real-time-systems content — this is control engineering you already know.

What Phase 0 is — and is not. You are not producing a research result; you are producing the *machine that will produce every research result for the next six months*. That changes the quality bar in a specific way: cleverness matters less, and reproducibility, documentation, and interface discipline matter much more. Every shortcut you take here (hard-coded paths, undocumented parameters, controller logic tangled into the plant model) will be paid back with interest in Phases 1–5. The acceptance test is literal: a clean clone of your repository must regenerate the baseline figure with one script — because that exact test, run by the advisor on a machine you have never touched, opens the September checkpoint (Section 8, checklist item 1).

0.1 Environment and repository. Install MATLAB/Simulink with the Control System Toolbox, MPC Toolbox and/or ACADOS (your call which solver backend to start with — see 0.3b), and Simulink Coder. Install **MATLAB/SIMULINK SoC Blockset**. Create the Git repository with one folder per phase (phase0/ … phase6/), a common/ folder for the plant wrapper and controller family, and a top-level README that a stranger could follow to run everything. Commit early and often; tags mark milestones.

0.2 Plant integration (read-only rule applies). Integrate the mid-fidelity Simulink vehicle model (from the Veh Dyn Ctrl course) as the plant. Wrap it in a subsystem with a documented I/O interface: inputs (steering, drive/brake commands), outputs (the measured states the controller will consume), units and sign conventions written into the README. Then run two open-loop sanity checks and record the plots: (a) steady-state speed vs. constant throttle (does the drag balance look physical?), (b) a step-steer response at constant speed (does the yaw response look like a car?). Disturbance inputs you'll need later (task 3.5) get added *in your wrapper*, not inside the model.

0.3 MPC baseline at Ts = 10ms. Longitudinal speed tracking and lateral path tracking; whether combined in one MPC or split into two loops depends on the plant's interface. Two hard requirements:

- **(a) Swappable controller module.** The controller lives in a self-contained subsystem with a fixed I/O signature (states in, control commands + solve-time out), so the solver backend (MPC Toolbox vs. an ACADOS S-function) can be swapped without touching anything else.
- **(b) Deliberately low-order internal model.** The MPC's prediction model can be simplified, *even though the plant is mid-fidelity*. This is intentional, not a limitation: no production MPC predicts with a full vehicle model, and the plant–model mismatch is part of what we are studying. Do not improve the internal model too much to chase tracking performance.

Tune once on the nominal scenario, document weights, horizon (record it **in seconds**, not steps — Phase 3 re-discretizes at other rates and the horizon-in-seconds is what stays fixed), and constraints in the README, then **freeze** (working rule 4).

0.4 Scenario library. Define three reference scenarios, reused unchanged for the entire study: (i) **highway cruise** — near-constant speed, gentle curvature; (ii) **urban profile** — stops, speed changes, moderate turns; (iii) **aggressive maneuver** — double lane change or braking-while-steering. Implement each as a data file plus a loader function, not as blocks scattered in the model, so Phase 3 can later attach a demand tag to the same files. Verify closed-loop tracking on all three; report RMS and peak errors per scenario in the memo.

0.5 Solver-time telemetry (start now). Log the MPC solve time at every control step, in every run. For Phase 0 specifically: run all three scenarios and produce a solve-time histogram per scenario, plus a small table of mean / 95th percentile / max.

Expected observation, worth a sentence in the Report: solve time is *scenario-dependent*, because constraint activity changes how hard the QP/NLP is — the aggressive maneuver should be visibly more expensive than cruise. That scenario-dependence is not a nuisance; it is the first genuinely novel ingredient of this project, because in Phase 2 this measured distribution *becomes the control task's execution-time model*, replacing the made-up constants older studies used.

0.6 Reading and memo. Read items 1–2. Report #1 contains: the Chang's PhD dissertation summary and research gap map, the CDC-2000 paper summary (main idea and your own questions), the baseline tracking figure, and the solve-time table.

Deliverables: repository with plant wrapper + frozen MPC + scenario library, baseline tracking figure, solve-time histograms and table, Report #1.

Acceptance: clean clone reproduces the baseline figure with one script in under ten minutes.

5. Phase 1 — Timing Sensitivity Study (July) → Figure P1

Objective. Build the quantitative map from timing non-ideality to control degradation — empirically by simulation, and analytically by eigenvalue analysis — and show the two agree. This is the core preliminary evidence for Research Question 1, and Figure P1 is checklist item 2 at the checkpoint.

What Phase 1 is — and is not. There is no scheduler yet and no CPU model. Delay in this phase is injected *artificially* with a delay block, precisely so you can control it as a clean independent variable and sweep it. Phase 2 then replaces the artificial delay with delay generated by a real cause; the degradation map you build here is the yardstick that makes Phase 2's results interpretable.

The mechanism. Delay is phase lag: the plant receives a command computed for where the state *was*, not where it *is*. In feedback terms, delay erodes phase margin; in state-space terms, the delayed loop has extra eigenvalues that migrate toward the unit circle as the delay grows. Both views describe the same physics, and task 1.4 makes the second one quantitative.

1.1 Constant actuation delay. Insert a delay block between the controller output and the plant input (inside your wrapper, so the controller module stays untouched). Sweep τ from 0 to 100ms in 5ms steps, on scenarios (ii) and (iii).

Metrics per run: tracking RMS error, peak lateral error, constraint-violation count. Plot metric vs. τ; identify two landmarks — the **knee** (where degradation stops being graceful) and the **destabilizing delay** (defense: define your instability criterion numerically, e.g., error exceeding a bound or divergence, and state it in the caption). Practical note: implement the sweep as a script looping with the delay as a parameter — no hand-editing the model between runs, or the sweep is not reproducible.

1.2 Random delay. Per-step delay drawn from the uniform distribution $U(0, \tau max)$. Sweep τmax over the same range; 50 Monte Carlo repetitions per value with fixed, logged seeds; plot mean tracking RMS with $\pm 1\sigma$ shaded bands vs. τmax. Sanity check: the τmax $\rightarrow$ 0 limit must land on the 1.1 curve's origin.

1.3 Sampling jitter. Now perturb *when the controller runs* instead of when the command arrives: actual sampling instants $T_s + \delta$, $\delta \sim U(-J, J)$, J swept 0 to 5 ms. (Implementation hint: drive the controller subsystem as a triggered subsystem from a script-generated event sequence, rather than fighting Simulink's fixed-step scheduler.) Same metrics and Monte Carlo protocol. Then answer in the Report: is this loop more sensitive to *mean delay* or to *jitter of the same magnitude*? (Think about what the ZOH between samples does to each.)

1.4 Analytical cross-check. This subtask has four concrete steps:

1. **Extract the implied linear gain.** In the unconstrained regime your MPC is a linear state feedback $u = -K \cdot x$. Get K either by solving dlqr on the MPC's internal model with matched weights, or numerically: perturb each state around the operating point, record the MPC's response with constraints inactive, and fit K.
2. **Build the augmented closed loop.** For a delay of n whole periods, the applied input is $u(k) = -K \cdot x(k-n)$. Augment the state $z(k) = [x(k), u(k-1), \ldots, u(k-n)]$ so the loop becomes $z(k+1) = A_{cl}(n) \cdot z(k)$; assemble $A_{cl}(n)$ in a loop over n (block structure: plant matrices on top, the gain writing into the newest input slot, a shift chain below).
3. **Compute the spectral radius.** $\rho(n) = \max(\text{abs}(\text{eig}(A_{cl}(n))))$ for n = 0, 1, 2, …; stability iff $\rho < 1$. Plot ρ vs. $n \cdot T_s$ in milliseconds.

4. **Overlay and explain.** Mark where ρ crosses 1 (the *predicted* destabilizing delay) against the *empirical* instability point from 1.1. Then mark where the two curves diverge and explain why: during large maneuvers constraints activate, the MPC stops being the linear gain −K, and the linear prediction loses validity exactly there.

Figure P1 (deliverable): two panels — (a) tracking RMS vs. delay with Monte Carlo bands; (b) spectral radius vs. delay with the unit-circle crossing marked against the empirical instability point, and the constrained-regime divergence annotated. **Report #2** (two pages): what you observed, what surprised you, the mean-delay-vs-jitter argument, and how readings 2–3 explain the mechanisms.

6. Phase 2 — Scheduler-in-the-Loop: Testbed v1 (August) → Figure P2 (draft)

What Phase 2 is — and is not. You are NOT building a vehicle perception or planning system. There are no cameras, no sensor models, no object detection, no Autoware-style stack. The controller keeps reading its states directly from the vehicle plant, exactly as in Phases 0–1. The "perception task" below is **synthetic CPU load**: a task that computes nothing anyone uses and produces no signal. It is fully described by three numbers — a period, an execution time, and a priority — because that is all a scheduler ever sees of any task. Its only job is to occupy the CPU so the control task gets preempted and delayed. One sentence to remember: *perception is modeled as CPU demand, not as function.*

The mechanism you are building. Four links, wired in order:

1. The load task occupies the CPU for part of every 50 ms window
2. Because of preemption and queueing, the control task's **response time** (release → completion) stretches beyond its ~2 ms solve time — sometimes past its 10 ms deadline.
3. Each control cycle, that response time is applied to the *real* closed loop as that step's actuation delay; a missed deadline means the plant **holds the previous input** for that cycle (which is what real automotive middleware does).
4. Delayed and held inputs degrade tracking

The **scheduler simulation** and the **vehicle simulation** are two worlds connected by one signal: the per-cycle response time. That is the entire coupling.

2.1 Tool spike. Goal: demonstrate the four-link chain with a trivial toy — one control task + one load task, response time routed into a delay block, any plant (even a double integrator is fine).

Primary route: **MATLAB/SIMULINK SoC Blockset** Task Manager — it natively models task execution times (including sampled from data, i.e., your Phase-0 solve-time measurements) under preemptive fixed-priority scheduling inside a normal Simulink simulation. If SoC Blockset, a possible fallback is **SimEvents**. Do not integrate with the real testbed until the toy example works.

2.2 Task set (the standard configuration for every Phase-2+ experiment). Three periodic tasks on one simulated CPU, preemptive fixed-priority scheduling, priorities assigned by rate (shorter period = higher priority — mirroring AUTOSAR/OSEK practice):

Task	Period	Execution time	Deadline	Priority
Communication	5ms	0.5ms (fixed)	5ms	High
Control (MPC)	10ms	**sampled from your Phase-0 measured solve-time distribution**, per scenario	10ms	Medium
Perception-like load	50ms	**10–45ms — the experiment knob**	50ms	Low

Note the asymmetry in care: the control task's execution time uses *measured* data because its latency feeds the loop; the other two are stylized because only their CPU footprint matters.

2.3 Integration and sanity checks. Drop the scheduler module into the testbed: control-task response time → per-step actuation delay; deadline miss → hold previous input. Two checks before any sweep, commit only after both pass:

(a) **load off** — with the perception task disabled, Phase-2 tracking must reproduce the Phase-0 baseline;

(b) **load maxed** — response-time traces must visibly show the control task being preempted and stretched. If (a) fails, your coupling path is leaking delay somewhere even when there is none; find it immediately.

2.4 The experiment. Sweep the load task's execution time so total utilization U = Σ(execution/period) goes from 0.5 to 1.2, on scenarios (ii) and (iii). Per sweep point, log: control-task response-time histogram, deadline-miss count and pattern,

tracking RMS error. Expect the interesting region near U ≈ 0.9–1.0 where response times explode — sample it densely. And connect back to Phase 1: at each utilization point you now know the *realized* delay distribution, so you can ask whether the Phase-1 map (delay → degradation) predicts the Phase-2 outcome (utilization → delay → degradation). One paragraph on that in the Report turns two figures into one coherent story.

Figure P2 draft (deliverable): tracking cost vs. CPU utilization under fixed-priority scheduling, deadline-miss onset annotated; inset: response-time histogram at U = 0.9. **Report #3** must explain in your own words why response time explodes as U approaches 1 (Buttazzo Ch. 1–4 is the companion reading).

Testbed v1 definition (checkpoint artifact). One Simulink model plus configuration scripts in which plant, controller, scheduler, and workload profile are separately swappable modules, and one script regenerates the P2 draft from a clean clone. Version 1 must *run and be reproducible*, and the clean-clone regeneration test described there is exactly how it will be judged.

Note that in a real vehicle, perception load would also degrade the *state estimate* the controller consumes (stale detections, delayed localization). Our testbed deliberately omits that coupling — the controller reads plant states directly — and models perception purely as CPU demand. Write this limitation into Report #3.

7. Phase 3 — Naive Open-Loop Adaptive-Sampling Supervisor (August to early September) → Demo Figure S0

Objective. A deliberately simple existence proof: adapting the control sampling period to the driving situation frees CPU capacity at small tracking cost. This is a checkpoint demo, not a proposal figure, and not the research contribution — the rigorous, feedback version is Phase 5.

To be clear, the supervisor you build here is **open-loop**: it selects the sampling period from the *reference schedule known in advance*. It does NOT look at the measured tracking error, does NOT look at CPU utilization or temperature, and comes with NO stability or performance guarantees. Building it naive is the point: its documented failures write the requirements for the real supervisor. Budget accordingly — the switching rule itself should be under ~20 lines of logic; most of your time goes into making variable-rate execution work cleanly inside the testbed.

The control task's CPU demand is execution-time/period. Its execution time (the MPC solve) is what it is — but the **period is a design variable**. Cruising on the highway, the reference barely changes between 10ms samples; running the controller at 50ms costs little tracking and cuts the control task's CPU demand by 5 times. During a lane change, 50ms sampling would be dangerous, so the supervisor switches back to 10ms. The freed CPU time is headroom for the perception load, which is exactly the resource-management story of the whole project, in its simplest possible form.

3.1 Segment tagging. Extend the scenario library: each scenario definition gets a demand signal over time, computed offline from the reference itself — e.g., preview window of 1s; if path curvature or commanded speed change in the window exceeds a high threshold → demand = high; above a low threshold → medium; else low. Map: high → Ts = 10 ms, medium → 20 ms, low → 50 ms. Add a **minimum dwell time** (e.g., 0.5 s) so the mode can't chatter. Plot the demand signal over each scenario and eyeball it before wiring anything — the tags should match your intuition of where the maneuvers are.

3.2 Variable-rate controller (the real work, and a rule amendment). An MPC's internal prediction model is discretized at a specific Ts, so one frozen controller cannot simply run at three rates. The clean solution: build **three controller instances** — the frozen Phase-0 MPC re-discretized at 10, 20, and 50ms with the *same* weights, horizon-in-seconds (so the horizon covers the same physical time: e.g., 20 steps at 10ms, 10 steps at 20ms, 4 steps at 50ms), and constraints. Verify each instance tracks acceptably at its own fixed rate on the cruise scenario, then **freeze all three**; this triple replaces the single frozen controller in the working rules from here on. Switching happens only at the completion of a control cycle; between samples the plant holds the last input (ZOH), which the testbed already does.

3.3 Scheduler hookup. In testbed v1, the control task's period now follows the supervisor's mode. Utilization consequence to verify explicitly: at Ts = 50ms the control task's demand drops ~5×, so total utilization at fixed perception load must drop accordingly. Check before any experiment: force demand = high everywhere — the run must reproduce the Phase-2 fixed-10ms result. Commit only after this passes.

3.4 The demo experiment. Scenario (i) or (ii), fixed perception load at a level that is comfortable at Ts = 10ms. Two runs: fixed 10ms vs. the naive supervisor. **Demo Figure S0:** three stacked panels sharing the time axis — (top) reference speed/path with the Ts mode trace overlaid as a colored band; (middle) CPU utilization, both runs; (bottom) tracking error,

both runs. Report two headline numbers in the caption: average utilization reduction (%) and tracking RMS degradation (%). The story you want visible at a glance: utilization drops a lot, tracking degrades a little.

3.5 The engineered failure. Add one run where something happens that the reference schedule doesn't know about — e.g., a disturbance (wind gust) hitting during a low-demand, 50ms segment. The open-loop supervisor keeps sampling slowly while the error grows; the fixed-10ms baseline recovers faster. Put this run in Report #4.

Report #4 (the deliverable that matters most here, ~1.5 pages): (a) what the naive rule achieves and the two headline numbers; (b) the failure analysis — precisely *why* the open-loop rule fails in 3.5 (it cannot sense what it does not measure); (c) therefore, the requirements list for the Phase-5 closed-loop supervisor: which signals it must sense (tracking error, disturbance activity, utilization, temperature margin) and what it must trade off.

8. Checkpoint — First Week of September (Go/No-Go for the Studentship)

Logistics. We will hold a ~90-minute review meeting in the first week of September.

One working day before the meeting, send: (1) the Git tag checkpoint-v1 on the repository, (2) PDF copies of Figure P1, Figure P2 draft, and Demo S0, (3) Reports #1–#4 collected into one file. Before the meeting, I will clone the tagged repository and run the figure-regeneration scripts; the meeting starts from whatever that clean-clone run produced.

Deliverable checklist. Each item has a *minimum bar* (required for a Go) and a *strong* level (what excellent looks like). Stretch items (1.5, 2.5, lateral variants) are explicitly **not** on this list and their absence costs nothing.

#	Artifact	Minimum bar	Strong
1	Testbed v1	Clean clone regenerates the P2 draft with one script, in under ~30 min	Modules genuinely swappable; README lets a stranger run any experiment
2	Figure P1	Empirical sweep correct; spectral-radius curve computed; the two roughly agree	Crossing points match closely; the constrained-regime divergence is marked and explained
3	Figure P2 draft	Utilization sweep runs; degradation visible; miss onset annotated	Response-time histograms clean; dense sampling near $U \approx 1$; measured solve-time distribution demonstrably in use
4	Demo S0	Supervisor runs; utilization visibly drops; tracking loss quantified	Engineered-failure run analyzed; dwell logic clean; headline numbers in caption
5	Memos #1–#4	On time, complete, readable	Sections quotable in a proposal, especially Memo #4(c)

How the discussion will go. Expect to answer, at a whiteboard, without notes: *Why does actuation delay destabilize a feedback loop — what is the mechanism, not just the observation? Why does the control task's response time explode as utilization approaches 1? Where does the spectral-radius prediction stop being valid for your MPC, and why exactly there? What must the closed-loop supervisor sense that the naive one cannot, and what new risk does closing that loop introduce?* The readings plus your own experiments contain every answer.

Decision criteria (weighted roughly equally):

- **Technical quality.** Do the artifacts meet the minimum bars, and are they *correct* — not just plausible-looking?
- **Reasoning.** The whiteboard questions above. Fluent mechanism-level explanations in your own words, not recited paper phrases.
- **Ownership.** The open question is whether you drive research: did you make sensible calls in ambiguity, notice and chase anomalies, propose next steps unprompted, and treat this as your project rather than a task list?
- **Judgment under blockage.** Did you solve routine problems yourself and escalate the right ones (tool spike outcome, licensing, plant-model issues) at the right time, with precise problem descriptions?

On honesty about problems. If something did not work, a documented failure with a correct analysis of *why* counts **in your favor** at this checkpoint — that is what research looks like. A polished figure hiding an unexamined anomaly counts against you. When in doubt, show the anomaly.

Possible outcomes.

Go: funded studentship from September; proceed to Phases 4–6.

No-go: we will discuss it candidly; the codebase and memos remain yours to show as portfolio work.

Priorities if time runs short (in order): a reproducible testbed and a correct Figure P1 outrank everything; then the P2 draft; then S0. Depth beats breadth — do not thin all four to finish all four.

9. Real-Time Concepts Cheat Sheet for a Control Engineer

Term	One-line meaning
Task / period / deadline	A recurring job released every period that must finish by its deadline; your controller is a task with period Ts
WCET	Worst-case execution time — the upper bound on how long one job takes; timing analysis uses WCET the way robust control uses worst-case uncertainty
Response time / latency	Release-to-completion time of a job; this is the input delay your Phase 1 study injects
Jitter	Cycle-to-cycle variation of the response time; a stochastic perturbation of the sampling process
Rate Monotonic (RM)	Fixed priorities: shorter period = higher priority; simple, but guarantees full utilization only up to ~69% for many tasks
EDF	Dynamic priorities: earliest deadline first; schedulable up to 100% utilization, but degrades unpredictably when overloaded
Utilization U	Sum of execution-time/period ratios across tasks; U > 1 means demand exceeds the CPU
Preemption	A higher-priority task interrupts a running lower-priority one — the mechanism that turns a shared CPU into delay and jitter
Weakly-hard (m,K)	At most m deadline misses in any K consecutive periods
DVFS	Dynamic voltage/frequency scaling — the chip trades speed for power; dynamic power scales roughly with f^3
Thermal throttling	The hardware's own protective feedback loop: temperature above threshold forces the frequency down, stretching every execution time

10. Common Pitfalls

- **Do not retune the MPC per experiment** — weights, horizon, and internal model are frozen after Phase 0; if a run misbehaves, that is a result, not a tuning bug.
- **Do not upgrade the plant mid-study** — the mid-fidelity model is frozen infrastructure; fidelity creep silently invalidates earlier figures.
- **Do not conflate solver failure with deadline miss** — log infeasible/max-iteration solver exits separately from scheduling-induced misses; they are different failure mechanisms and the proposal will discuss them differently.
- **Do not gold-plate testbed v1** — modular and reproducible beats elegant; refactoring is a fall activity.
- **Do not present a figure you cannot regenerate.** Fixed seeds, one script per figure, committed before the memo goes out.